Human in the Loop for Fuzz Testing: Literature Review and the Road Ahead

JIONGCHI YU*, Singapore Management University, Singapore
XIAOLIN WEN*, Nanyang Technological University, Singapore
XIAOTIAN ZHAO, Nanyang Technological University, Singapore
JIAJUN HUANG, Nanyang Technological University, Singapore
XIAOFEI XIE, Singapore Management University, Singapore
YONG WANG, Nanyang Technological University, Singapore

Fuzz testing has emerged as an effective method for software security analysis, yet its effectiveness often hinges on automated heuristics that struggle with complex vulnerabilities, which require human expertise for effective testing. This paper presents a forward-looking analysis of incorporating Human in the Loop (HITL) into fuzz testing, aiming to explore how human involvement can enhance fuzzing frameworks. With the help of *Visualization*, which displays fuzzing processes in a visual format, and *Human-Computational Interaction (HCI)* methods that allow for on-the-fly human interventions in the fuzzing process, HITL introduces critical improvement into key fuzzing stages such as instrumentation, seed selection, and mutation. We comprehensively discuss existing research on adopting HITL in fuzz testing across various stages of the fuzzing process and propose a detailed research agenda emphasizing (1) advanced visualization techniques for real-time fuzzing analytics and (2) adaptive HCI frameworks for effective fuzzing. We aim to inspire a paradigm shift toward interactive, human-guided fuzzing systems that are equipped to tackle persistent and complex challenges in software testing. Furthermore, in light of the advancements in Large Language Models (LLMs), this paper seeks to provide a strategic discussion regarding integrating human expertise with AI-powered automation within the next-generation fuzzing ecosystem.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; • **Human-centered computing → Information visualization**; **Human computer interaction (HCI)**.

Additional Key Words and Phrases: Fuzzing, Visualization, Human-Computer Interaction

ACM Reference Format:
Jiongchi Yu, Xiaolin Wen, Xiaotian Zhao, Jiajun Huang, Xiaofei Xie, and Yong Wang. 2018. Human in the Loop for Fuzz Testing: Literature Review and the Road Ahead. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

*Both authors contributed equally to this research.

Authors' Contact Information: Jiongchi Yu, Singapore Management University, Singapore, Singapore, jcyu.2022@phdcs.smu.edu.sg; Xiaolin Wen, xiaolin004@e.ntu.edu.sg, Nanyang Technological University, Singapore, Singapore; Xiaotian Zhao, xiaotian001@e.ntu.edu.sg, Nanyang Technological University, Singapore, Singapore; Jiajun Huang, jiajun006@e.ntu.edu.sg, Nanyang Technological University, Singapore, Singapore; Xiaofei Xie, xfxie@smu.edu.sg, Singapore Management University, Singapore, Singapore; Yong Wang, yong-wang@ntu.edu.sg, Nanyang Technological University, Singapore, Singapore.

Manuscript submitted to ACM

1 Introduction

Fuzz testing is a dynamic software testing technique that subjects a program to random inputs to identify anomalies like crashes and other security threats. This method aims to uncover software bugs that could be exploited by malicious entities, thus enhancing the security and reliability of the software. Since its concept is introduced in the 1990s [33], fuzz testing has gained significant traction in both industrial [1, 4] and academic circles [5, 9], evolving considerably in terms of effectiveness and efficiency.

Modern fuzzing workflows [48] can be deconstructed into five iterative phases, including:

- **Instrumentation**: Embedding feedback mechanisms (e.g., coverage tracking) into target programs.
- **Initial Seed Selection**: Curating initial test inputs that bootstrap meaningful exploration.
- **Search Strategy & Power Scheduling**: Prioritizing seeds with high potential to trigger novel software behavior.
- **Mutation**: Applying transformations of the fuzz corpora to generate new test cases.
- **Execution & Monitoring**: Detecting crashes, hangs, or security-relevant anomalies.

The significance of fuzz testing lies in its ability to uncover vulnerabilities that traditional testing methods such as unit testing may overlook. Conventional testing approaches often rely on predefined test cases, which can be limited in scope and may not account for the myriad of ways in which a program can be misused. Fuzzing, on the other hand, generates a wide range of inputs, allowing for a more exhaustive exploration of the program's behavior. This capability is particularly crucial in the context of security, where even a single vulnerability can lead to catastrophic consequences. As such, fuzz testing has gained recognition as an essential practice in the software development lifecycle. While automated tools excel in brute-force input generation, they falter in scenarios requiring semantic awareness [8, 42] (e.g., generating inputs that satisfy protocol state machines) or strategic prioritization [17, 25]. For instance, a fuzzer targeting a TLS parser may waste cycles mutating packet headers while overlooking cryptographic handshake logic—a gap well-understood by human experts but opaque to coverage heuristics. To enhance fuzzing performance in specific domains, existing research explores various domain-specific fuzzing techniques, such as for web applications and protocols [2, 21, 49], operating systems [6, 14, 19], and deep learning systems [18, 45, 46].

The Need for Human-in-the-Loop. Despite its effectiveness, fuzz testing still faces challenges. Existing research [40, 48, 51] has shown that the performance of fuzzing greatly depends on the developer's understanding of security and code, with efficient fuzzing tools strongly depending on domain expertise from the security domain, which lacks generalization ability. The sheer volume of data generated during fuzz testing can be overwhelming, making it difficult for testers to discern meaningful insights from the noise. Additionally, automated fuzz testing tools [23, 35] often lack the contextual understanding that human testers possess, leading to missed opportunities for identifying vulnerabilities. For example, testers of fuzzing suites often have less domain knowledge compared to developers, which can render them less effective when developing fuzz testing for complex and semantically rich software, such as networking protocols or large system-level software [27, 28, 31, 32]. However, for developers, they often lack testing expertise, who often struggle to contribute in enhancing fuzz testing. These challenges are exacerbated in complex, semantically rich domains like network protocols or machine learning frameworks, where vulnerabilities often reside in logical flaws (e.g., authentication bypasses) rather than memory errors. Here, human expertise becomes irreplaceable as security analysts can guide instrumentation toward critical functions, curate seeds that reflect adversarial intent, and interpret intermediate results to adjust strategies.

To operationalize human expertise, existing research mainly adopt two complementary approaches to enhance fuzzing effectiveness:

- **Visualization for Fuzzing**: Existing research such as V-Fuzz [24] and VisFuzz [51] transform raw fuzzer outputs (e.g., coverage maps, crash patterns) into interactive dashboards, enabling analysts to identify under-tested code regions or anomalous patterns. For example, heatmaps overlaying coverage data with code complexity metrics allow the testers to apply prioritized testing strategies which focuses on high-risk, low-coverage functions.
- **Human-Computational Interaction (HCI) for Fuzzing**: Fuzzing frameworks such as Hoedur [38] and AFLTriage [16] integrate real-time human interventions directly into fuzzing loops, e.g., annotating crashes, adjusting mutation weights, or injecting domain-specific rules. These tools incorperate human intuition and excels in high-level strategy while machines handle repetitive tasks compared to traditional fuzzing frameworks.

New Era with Large Language Models Recent advancements in artificial intelligence, particularly with Large Language Models (LLMs), have introduced new opportunities to enhance fuzz testing frameworks [12, 30, 31, 43, 44]. Current research has explores LLMs into the fuzzing workflow to leverage domain-specific knowledge and improve the efficacy of fuzz testing. However, the development and application of LLMs still necessitate human expertise. The incorporation of LLMs has significantly transformed the HITL paradigm for fuzzing, presenting novel opportunities and challenges. This integration not only augments the fuzzing process with advanced predictive capabilities but also redefines the interaction between human operators and automated testing systems, opening avenues for more sophisticated, intelligent testing strategies.

Toward Collaborative Fuzzing Paradigm This vision paper provides a comprehensive overview of the current state of HITL fuzzing, emphasizing the critical role of human involvement in enhancing the effectiveness of fuzz testing across various stages and rethinks the relationship between automated testing frameworks, human efforts and the rising AI models. By synthesizing existing literature and outlining a research agenda, we aim to facilitate future advancements in this vital domain. This approach not only highlights the indispensable contributions of human expertise but also sets the stage for developing more integrated and effective fuzz testing methodologies.

2 Review Methodology

To provide a comprehensive understanding of existing approaches using HITL methods for fuzz testing, we followed established literature review practices [39, 50, 52] and summarized research based on different stages of modern fuzzing frameworks [22]. We categorized the fuzzing process into five key stages: **Instrumentation**, **Initial Seed Selection**, **Search Strategy & Power Scheduling**, **Mutation**, and **Execution & Monitoring**.

2.1 Paper Collection

For the collection of research papers, all authors jointly searched using specific keywords, including *HCI*, *Visualization*, and *Fuzz Testing* from top-tier conferences and journals published over the past ten years, engaging in discussions to determine their relevance to the HITL fuzzing paradigm. Our criteria for inclusion focused on instances where human intervention directly enhances the fuzzing process at any of these stages, such as modifying mutated seeds or analyzing visualized data to identify bottlenecks. Ultimately, we identified 61 relevant research papers.

2.2 Paper Selection

Inclusion and Exclusion Criteria. After the papers are collected, we include the following rules based on discussion in the criteria for whether the papers are selected:

✔ The paper must be in English.

✔ The selected papers must be directly related to fuzzing and explicitly include aspects of human involvement in the fuzzing process.

✗ The paper has less than 4 pages.

✗ Books, keynote records, panel summaries, technical reports, theses, tool demos papers, editorials, or venues not subject to a full peer-review process.

✗ Just mentioning fuzzing without human involvement.

We conduct a manual annotation study and filtering process to identify papers on human involvement in fuzzing. Initially, all authors individually read and categorized the papers. Subsequently, we collectively discuss the categorizations and address any discrepancies. In the second round, we revisit only the 12 papers categorized differently by various authors to ensure consistency. Afterward, we focus on discussing these papers and refine our selection to include only those explicitly involving human participation, excluding those centered on fully automated fuzzing tools. Ultimately, we identify 26 papers that explicitly involved human participation in the fuzzing process through manual review.

3 Current Development

The integration of human expertise into fuzzing workflows has evolved from ad-hoc manual interventions to systematic, tool-supported methodologies. This section critically examines state-of-the-art HITL fuzzing approaches across the five core phases during fuzzing, emphasizing how visualization and interactive mechanisms bridge human intuition with automated exploration.

3.1 Instrumentation

Instrumentation is critical in fuzzing campaigns as it determines what run-time data are collected, such as code coverage and memory access patterns. While automated tools like AFL++ rely on static coverage targets, recent advancements allow experts to prioritize security-critical regions. Current research has largely focused on developing visualization-driven instrumentation approaches. For instance, FuzzPlanner [10] features a multi-view dashboard, including timelines and binary graphs, to visualize firmware emulation workflows. Analysts can manually annotate high-risk binaries, such as network stacks, to refine the focus of instrumentation. Similarly, FuzzInspector [26] displays code coverage heatmaps alongside function-level complexity metrics, enabling users to pinpoint under-instrumented modules like cryptographic libraries with insufficient coverage. However, a significant limitation of current tools is their lack of real-time adaptability; once set, instrumentation adjustments typically require restarting the fuzzing campaigns. This necessitates further research into developing more dynamic instrumentation methods that can adapt on the fly without disrupting ongoing processes.

3.2 Initial Seed Selection

High-quality seeds are crucial for initiating effective fuzzing campaigns. Automated seed generators typically produce syntactically valid inputs but often lack semantic depth, making human curation necessary. In terms of seed selection, existing research predominantly focuses on human-interactive methods to enhance the quality of fuzzing. For instance, Infuzz [47] visualizes seed execution traces as call graphs, pinpointing seeds that traverse security-sensitive APIs such as malloc and memcpy. Experts then prune seeds that redundantly cover trivial paths. Similarly, DEFT [37] utilizes log analysis to extract protocol state transitions from 5G test data, allowing users to interactively select log segments as seeds that adhere strictly to protocol semantics. Interestingly, the research also emphasizes exploring the diversity of seeds. Dynodroid [29] allows users to pause fuzzing to provide domain-specific seeds and then resume the process to

seamlessly blend manual insight with automated test generation. Grishin et al. [15] demonstrate that expert-selected seeds achieve 2.3× faster coverage growth in network protocol fuzzing compared to seeds from automated corpora. However, current research acknowledges limitations; notably, an over-reliance on human-curated seeds risks biasing the exploration towards known attack patterns, potentially overlooking novel vulnerabilities.

3.3 Search Strategy & Power Scheduling

This phase of fuzzing strategically allocates energy to seeds that are most likely to uncover new behaviors. While automated schedulers typically prioritize gains in coverage, human insight can focus on exploring high-risk paths. Existing work primarily emphasizes visualizing this process to aid in fuzzing decisions. For example, FuzzSplore [13] uses scatter plots to correlate seed energy allocation with code complexity metrics. Analysts can manually deprioritize seeds that are stuck in low-risk loops, such as parsing trivial headers. Conversely, DEFT [37] incorporates machine learning models to predict vulnerabilities in protocol layers. Users can adjust model confidence thresholds to strike a balance between minimizing false positives and avoiding missed detections. Additionally, HaCRS [40] demonstrates how human operators can overcome 'semantic barriers' by steering the fuzzer towards inputs that satisfy specific protocol state machines, like valid TLS handshakes. This approach has led to 68% higher path coverage in OpenSSL, showcasing the significant impact of integrating human expertise into the fuzzing process.

3.4 Mutation

Mutation strategies in fuzz testing vary widely, from simple random bit-flipping to more sophisticated grammar-aware transformations. Human expertise plays a crucial role in defining context-sensitive rules for these mutations. Notably, existing research has leveraged advancements in visualization and Human-Computer Interaction (HCI) to enhance the efficacy of mutation phases in fuzz testing. For instance, Visual-Driven Mutation Tuning involves techniques such as those employed by FuzzInspector [2], which visualizes constraint violations (e.g., failed checksums) as logic equations, enabling analysts to adjust mutation manually ranges to meet specific constraints, like ensuring valid header lengths. Similarly, FMViz [20] displays mutations as pixel matrices, helping users identify over-mutated regions (e.g., repetitive byte flips in HTTP headers) and apply structure-preserving rules.

HCI interventions further support the fuzzing process by enhancing mutation effectiveness. A notable example is Ijon [3], which allows experts to annotate program variables (e.g., packet_type) during execution. This information guides the fuzzer to mutate semantically coherent fields, such as swapping GET and POST in HTTP methods. However, despite these advances, there remains a lack of sufficient generalizability in the complex mutation rules employed by most advanced fuzzing frameworks, highlighting a need for more adaptable mutation strategies.

3.5 Execution & Monitoring

Post-execution analysis represents a critical phase where human expertise significantly influences the efficacy of fuzzing, especially in differentiating critical vulnerabilities from less significant findings. During this stage, visualizations frequently assist in monitoring the fuzz testing process. For instance, VisFuzz [51] utilizes time-series charts to detect testing stagnation, such as no new paths being discovered for over 24 hours. Analysts may respond by altering fuzzers or introducing new seeds. FuzzFactory [36] allows users to define custom predicates (e.g., is_leakage(input)) to track potential side-channel leaks, with violations prompting visual alerts for manual inspection.

Visualization methods also play a crucial role in crash diagnosis. For example, Fuzz Introspector [11] links crash data with code complexity metrics, such as cyclomatic complexity, to prioritize the review of crashes occurring in

high-risk functions. PrettiSmart [41] visualizes fuzzing-generated software behaviors to help verify software security. Furthermore, a study by Bundt et al. [7] demonstrated that Human-in-the-Loop (HITL) triage could reduce false positive rates by 41% in automotive CAN bus fuzzing when compared to automated classifiers.

In summary, current research on HITL for fuzzing highlights primary trends, including Phase-Specific Specialization and Reactive Over Proactive Interaction. Specifically, tools like FuzzInspector and VisFuzz exemplify Phase-Specific Specialization by targeting isolated phases without achieving end-to-end integration. In terms of interaction, there is a tendency for Reactive Over Proactive Interaction, where human interventions are typically triggered by failures rather than by anticipatory strategic adjustments.

Critical gaps identified in the field include the absence of a unified platform to support cross-phase HITL workflows, which would facilitate seamless adjustments in instrumentation based on triage findings. Additionally, significant latency-throughput trade-offs in tools like FuzzPlanner deter their use in large-scale campaigns. There is also a deficiency in evaluation metrics, with few studies quantifying the return on investment for human efforts in terms of hours spent versus vulnerabilities uncovered.

4 Vision

The evolution of HITL for fuzzing demands reimagining the roles of developers and security testers in vulnerability discovery. This section outlines a vision for next-generation fuzzing frameworks that harmonize domain expertise, interactive tooling, and AI-driven automation, tailored to the distinct needs of these two critical stakeholders.

4.1 For Developers

Developers, as architects of software systems, possess deep code knowledge but often lack specialized security training. Future HITL fuzzing frameworks must empower developers to seamlessly integrate security testing into their workflows, bridging the gap between code authorship and vulnerability mitigation. Specifically, they can focus on developing:

Context-Aware IDE Plugins. Integrate lightweight fuzzing assistants directly into IDEs for real-time context-driven fuzzing support. By analyzing code semantics and data flow, these tools suggest targeted instrumentation points, automatically generate initial test harnesses, and run fuzz tests in real time. Result visualizations like PrettiSmart [41] appear directly in the code editor, enabling developers to quickly identify issues and refine their code accordingly.

Automated Vulnerability Hypothesis Generation. Leverage LLMs trained on code-security correlations to automatically detect potential vulnerabilities or attack vectors. The system then provides interactive mutation previews that show how targeted input changes might trigger crashes or reveal other abnormal behaviors. This approach empowers developers to quickly evaluate AI-generated hypotheses, validate or discard them, and refine their code to address any real security concerns.

Optimized Seed Corpora. Implement auto-curation systems that learn from developer feedback. When developers triage crashes (e.g., marking a buffer overflow as critical), the system would autonomously refine mutation strategies to prioritize similar patterns in future campaigns.

Enabling developers to detect vulnerabilities during the coding phase, known as Shift-Left Security, can reduce remediation costs by 6080% compared to fixing issues post-deployment. Additionally, by using guided workflows, junior developers can achieve expert-level fuzzing efficacy, which democratizes access to advanced security testing.

4.2 For Security Testers

Security testers of software require tools that amplify their ability to uncover deep, context-dependent vulnerabilities while managing large-scale campaigns. Future systems must transform testers from passive observers to strategic directors of AI-powered fuzzing agents.

The main directions for security testers in enhancing HITL for fuzzing include:

Semantic Campaign Orchestration. Develop natural language interfaces (e.g., "Focus on authentication bypasses in the OAuth flow") that translate tester intent into fuzzer configurations. LLMs would decompose high-level objectives into actionable steps: augmenting seeds with session tokens, instrumenting authorization checks, and prioritizing crashes in token validation logic.

Adaptive Fuzzing Dashboards. Replace static visualizations with AI-powered visual analytics Dashboards capable of adaptively extracting critical information from vast streams of data generated by fuzzing campaigns. For instance, when examining an embedded TCP/IP stack, a tester could view real-time attack surface maps that highlight unexplored protocol states, guiding the fuzzer toward the most promising areas of investigation.

Collaborative Threat Modeling. Build shared knowledge bases where testers contribute protocol specifications, attack patterns, and triage insights. These would train domain-specific LLMs (e.g., ICS-FuzzGPT for industrial control systems) to generate semantically valid mutations and explain crashes in protocol-specific terms.

Our ultimate vision combines perspectives by establishing bidirectional feedback loops, ensuring that developers' identification of code hotspots informs tester prioritization and that testers' findings trigger automated code fix suggestions. We also integrate static code analysis, fuzzing coverage, and exploitability predictions into a unified risk matrix. This matrix, customizable for different roles, enables developers and testers to address vulnerabilities more effectively and efficiently.

5 The Road Ahead

5.1 Mutation Analytics

Understanding the effectiveness of mutation operators is critical for systematic vulnerability discovery. Existing approaches often treat mutations as opaque transformations with limited insight into their lineage or impact.

Phase 1: Mutation Lineage Benchmarking. We aim to construct a comprehensive benchmark that traces the "genealogy" of mutation operators across fuzzing campaigns. By instrumenting widely used fuzzers (e.g., AFL++, LibFuzzer), each mutated input can be linked back to the responsible operators (bit-flip, arithmetic insertion, etc.). By manually annotating crashes with vulnerability categories (e.g., buffer overflows, SQL injection), we can correlate operator lineage with the types of discovered bugs and analyze how operator effectiveness evolves (for instance, simple bit-flips might dominate in early exploration, whereas grammar-aware mutations could excel later). A visual analytics layer [22, 34] can further facilitate the discovery of the correlation between operator lineage and discovered bugs. The ultimate goal is to establish a publicly accessible dataset, tentatively named *MutBench*, which quantifies operator success rates across different vulnerability classes, thus allowing data-driven mutation strategies. The benchmark can serve as guidance for testers in fuzzing different software to choose their testing configurations.

Phase 2: Adaptive Mutation Strategy Learning. We further propose the development of interactive agents that dynamically select mutation operators based on real-time campaign analytics and are modifiable during the fuzzing phases for testers. By encoding fuzzing context during the fuzzing process, the representations can be visualized to testers and allow them to make decisions that prioritize vulnerability discovery rate over purely coverage-based metrics.

A human-in-the-loop mechanism would allow testers to override RL decisions (e.g., “suppress bit-flips in JSON fields”), thereby refining policy models through interactive feedback. We anticipate that such adaptive mutation strategies can significantly boost the discovery of critical bugs, with potential improvements of 30%-50% compared to state-of-the-art fuzzing methods.

5.2 LLM-Augmented HITL Fuzzing

Recent advances in LLMs present unprecedented opportunities for protocol comprehension and specification validation, especially in domains where expertise is scarce and costly.

Phase 1: LLM-Driven Specification Validation. We propose leveraging LLMs to extract and enforce protocol semantics during fuzz testing. By implementing retrieval-augmented generation (RAG) pipelines that ingest and interpret resources such as RFCs, API documentation, and network traces, it becomes possible to synthesize accurate protocol state machines (e.g., correctly identifying byte positions in HTTP/2 frames). Fine-tuned models such as CodeLlama can then generate mutations that preserve essential protocol invariants (e.g., modifying TLS cipher suites without invalidating the handshake). We also envision integrating these LLM-based validators into execution monitors, allowing them to flag or discard inputs that violate inferred specifications (e.g., a missing topic length field in MQTT messages). We aim to introduce a more tester-friendly user interface as well as the prioritized visualization platform to better allow testers to settle the fuzzing input specifications and format for achieving a higher quality of fuzzing. Ultimately, complex context-aware fuzzing tasks such as protocol fuzzers could maintain high syntactic validity while still exploring challenging semantic edge cases, thus reducing wasted fuzzing cycles by up to 60%.

Phase 2: Multi-Agent LLM Fuzzing Systems. We foresee the design of multi-agent LLM frameworks that emulate human roles at scale. Distinct LLM agent types would collaboratively manage diverse fuzzing tasks, such as generating input variations (Explorer Agents) or triaging crashes with vulnerability knowledge graphs (Analyst Agents). Strategist Agents could further optimize power schedules via the Monte Carlo tree search, informed by historical campaign data. In this way, the hard work for testers can be assigned to LLMs to assist in cope with the scenarios where testers are shortend, especially in open source communities. To incorporate human oversight, we propose interfaces that allow testers to issue natural language directives (e.g., "prioritize RCE vulnerabilities in the parser"), which the system translates into reward function updates. These architectures aim to deliver human-level triage accuracy at 100× the throughput of manual testing.

6 Conclusion

This paper advocates for a paradigm shift toward HITL for fuzzing, merging machine scalability with human expertise to tackle context-sensitive vulnerabilities. Short-term research will prioritize refining human-AI interaction through real-time visualization tools and adaptive interfaces, enabling experts to dynamically guide instrumentation, mutation, and triage. Simultaneously, we propose harnessing LLMs to automate semantic validation and enhance decision-making in complex fuzzing scenarios while addressing data confidentiality risks via robust access controls. Long-term goals focus on building self-adapting HITL systems capable of scaling across evolving software ecosystems, from IoT firmware to AI-driven applications. By systematically bridging automation gaps with human intuition and advancing LLM-augmented frameworks, this vision aims to establish fuzzing as a disciplined science of targeted vulnerability excavation, ultimately fortifying software security in an era of unprecedented complexity with the participation of human efforts.

References

[1] AFLplusplus. 2025. AFLplusplus. https://github.com/AFLplusplus/AFLplusplus/.

[2] Max Ammann, Lucca Hirschi, and Steve Kremer. 2024. DY fuzzing: formal Dolev-Yao models meet cryptographic protocol fuzz testing. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1481–1499.

[3] Cornelius Aschermann, Sergej Schumilo, Ali Abbasi, and Thorsten Holz. 2020. Ijon: Exploring deep state spaces via fuzzing. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1597–1612.

[4] Sadegh Bamohabbat Chafjiri, Phil Legg, Michail-Antisthenis Tsompanas, and Jun Hong. 2023. Honggfuzz+: Fuzzing by Adaptation of Cryptographic Mutation. *Phil and Tsompanas, Michail-Antisthenis and Hong, Jun, Honggfuzz+: Fuzzing by Adaptation of Cryptographic Mutation* (2023).

[5] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 1032–1043.

[6] Alexander Bulekov, Bandan Das, Stefan Hajnoczi, and Manuel Egele. 2023. No grammar, no problem: Towards fuzzing the linux kernel without system-call descriptions. In *Network and Distributed System Security (NDSS) Symposium*.

[7] Joshua Bundt, Andrew Fasano, Brendan Dolan-Gavitt, William Robertson, and Tim Leek. 2023. Homo in Machina: Improving Fuzz Testing Coverage via Compartment Analysis. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128.

[8] Luiz Carvalho, Renzo Degiovanni, Maxime Cordy, Nazareno Aguirre, Yves Le Traon, and Mike Papadakis. 2024. Specbcfuzz: Fuzzing LTL solvers with boundary conditions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[9] Peng Chen and Hao Chen. 2018. Angora: Efficient fuzzing by principled search. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 711–725.

[10] Emilio Coppa, Alessio Izzillo, Riccardo Lazzeretti, and Simone Lenti. 2023. FuzzPlanner: Visually Assisting the Design of Firmware Fuzzing Campaigns. In *2023 IEEE Symposium on Visualization for Cyber Security (VizSec)*. IEEE, 1–11.

[11] Ada Logics David Korczynski, Adam Korczynski. 2023. https://openssf.org/blog/2023/07/20/fuzz-introspector-optimizing-fuzzing-workflows/

[12] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.

[13] Andrea Fioraldi and Luigi Paolo Pileggi. 2021. Fuzzsplore: Visualizing feedback-driven fuzzing techniques. *arXiv preprint arXiv:2102.02527* (2021).

[14] Google. 2025. Syzkaller. https://github.com/google/syzkaller/.

[15] Maxim Grishin et al. 2022. Human-Controlled Fuzzing With AFL. (2022).

[16] Grant Hernandez. 2025. AFLTriage. https://github.com/quic/AFLTriage/.

[17] Muhammad Monir Hossain, Nusrat Farzana Dipu, Kimia Zamiri Azar, Fahim Rahman, Farimah Farahmandi, and Mark Tehranipoor. 2023. TaintFuzzer: SoC security verification using taint inference-enabled fuzzing. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.

[18] Li Huang, Weifeng Sun, Meng Yan, Zhongxin Liu, Yan Lei, and David Lo. 2024. Neuron Semantic-Guided Test Generation for Deep Neural Networks Fuzzing. *ACM Transactions on Software Engineering and Methodology* 34, 1 (2024), 1–38.

[19] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. 2021. Difuzzrtl: Differential fuzz testing to find cpu bugs. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1286–1303.

[20] Aftab Hussain and Mohammad Amin Alipour. 2021. Fmviz: Visualizing tests generated by AFL at the byte-level. *arXiv preprint arXiv:2112.13207* (2021).

[21] Samuel Jero, Maria Leonor Pacheco, Dan Goldwasser, and Cristina Nita-Rotaru. 2019. Leveraging textual specifications for grammar-based fuzzing of network protocols. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 9478–9483.

[22] Sriteja Kummita, Miao Miao, Eric Bodden, and Shiyi Wei. 2024. Visualization Task Taxonomy to Understand the Fuzzing Internals (Registered Report). In *Proceedings of the 3rd ACM International Fuzzing Workshop*. 13–22.

[23] Jun Li, Bodong Zhao, and Chao Zhang. 2018. Fuzzing: a survey. *Cybersecurity* 1 (2018), 1–13.

[24] Yuwei Li, Shouling Ji, Chenyang Lv, Yuan Chen, Jianhai Chen, Qinchen Gu, and Chunming Wu. 2019. V-fuzz: Vulnerability-oriented evolutionary fuzzing. *arXiv preprint arXiv:1901.01142* (2019).

[25] Jie Liang, Mingzhe Wang, Chijin Zhou, Zhiyong Wu, Yu Jiang, Jianzhong Liu, Zhe Liu, and Jiaguang Sun. 2022. Pata: Fuzzing with path aware taint analysis. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1–17.

[26] Han-Lin Lu, Ren-Jie Zhuang, and Shih-Kun Huang. 2023. Fuzz Testing Process Visualization. *Journal of Information Science & Engineering* 39, 5 (2023).

[27] Tao Lyu, Liyi Zhang, Zhiyao Feng, Yueyang Pan, Yujie Ren, Meng Xu, Mathias Payer, and Sanidhya Kashyap. 2024. Monarch: A Fuzzing Framework for Distributed File Systems. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 529–543.

[28] Xiaoyue Ma, Lannan Luo, and Qiang Zeng. 2024. From One Thousand Pages of Specification to Unveiling Hidden Bugs: Large Language Model Assisted Fuzzing of Matter {IoT} Devices. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4783–4800.

[29] Aravind Machiry, Rohan Tahiliani, and Mayur Naik. 2013. Dynodroid: An input generation system for android apps. In *Proceedings of the 2013 9th joint meeting on foundations of software engineering*. 224–234.

[30] Ruijie Meng, Gregory J Duck, and Abhik Roychoudhury. 2024. Large Language Model assisted Hybrid Fuzzing. *arXiv preprint arXiv:2412.15931* (2024).

[31] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, Vol. 2024.

[32] Ruijie Meng, George Pîrlea, Abhik Roychoudhury, and Ilya Sergey. 2023. Greybox fuzzing of distributed systems. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1615–1629.

[33] Barton P Miller, Lars Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Commun. ACM* 33, 12 (1990), 32–44.

[34] Tamara Munzner. 2009. A nested model for visualization design and validation. *IEEE transactions on visualization and computer graphics* 15, 6 (2009), 921–928.

[35] Olivier Nourry, Yutaro Kashiwa, Bin Lin, Gabriele Bavota, Michele Lanza, and Yasutaka Kamei. 2023. The human side of fuzzing: Challenges faced by developers during fuzzing activities. *ACM Transactions on Software Engineering and Methodology* 33, 1 (2023), 1–26.

[36] Rohan Padhye, Caroline Lemieux, Koushik Sen, Laurent Simon, and Hayawardh Vijayakumar. 2019. Fuzzfactory: domain-specific fuzzing with waypoints. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.

[37] Yifeng Peng, Xinyi Li, Sudhanshu Arya, and Ying Wang. 2023. Deft: A novel deep framework for fuzz testing performance evaluation in nextg vulnerability detection. *IEEE Access* (2023).

[38] Tobias Scharnowski, Simon Wörner, Felix Buchmann, Nils Bars, Moritz Schloegel, and Thorsten Holz. 2023. Hoedur: Embedded Firmware Fuzzing using Multi-Stream Inputs. (2023).

[39] Jieke Shi, Zhou Yang, and David Lo. 2024. Efficient and green large language models for software engineering: Vision and the road ahead. *ACM Transactions on Software Engineering and Methodology* (2024).

[40] Yan Shoshitaishvili, Michael Weissbacher, Lukas Dresel, Christopher Salls, Ruoyu Wang, Christopher Kruegel, and Giovanni Vigna. 2017. Rise of the hacrs: Augmenting autonomous cyber reasoning systems with human assistance. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 347–362.

[41] Xiaolin Wen, Tai D Nguyen, Lun Zhang, Jun Sun, and Yong Wang. 2024. PrettiSmart: Visual Interpretation of Smart Contracts via Simulation. *arXiv preprint arXiv:2412.18484* (2024).

[42] Feifan Wu, Zhengxiong Luo, Yanyang Zhao, Qingpeng Du, Junze Yu, Ruikang Peng, Heyuan Shi, and Yu Jiang. 2024. Logos: Log guided fuzzing for protocol implementations. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1720–1732.

[43] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2023. Universal fuzzing via large language models. *CoRR* (2023).

[44] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[45] Danning Xie, Yitong Li, Mijung Kim, Hung Viet Pham, Lin Tan, Xiangyu Zhang, and Michael W Godfrey. 2022. Docter: Documentation-guided fuzzing for testing deep learning api functions. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 176–188.

[46] Xiaofei Xie, Lei Ma, Felix Juefei-Xu, Minhui Xue, Hongxu Chen, Yang Liu, Jianjun Zhao, Bo Li, Jianxiong Yin, and Simon See. 2019. Deephunter: a coverage-guided fuzz testing framework for deep neural networks. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 146–157.

[47] Qian Yan, Huayang Cao, Shuaibing Lu, and Minhuan Huang. 2023. InFuzz: An Interactive Tool for Enhancing Efficiency in Fuzzing through Visual Bottleneck Analysis (Registered Report). In *Proceedings of the 2nd International Fuzzing Workshop*. 56–61.

[48] Qian Yan, Minhuan Huang, and Huayang Cao. 2022. A survey of human-machine collaboration in fuzzing. In *2022 7th IEEE International Conference on Data Science in Cyberspace (DSC)*. IEEE, 375–382.

[49] Xiaohan Zhang, Cen Zhang, Xinghua Li, Zhengjie Du, Bing Mao, Yuekang Li, Yaowen Zheng, Yeting Li, Li Pan, Yang Liu, et al. 2024. A survey of protocol fuzzing. *Comput. Surveys* 57, 2 (2024), 1–36.

[50] Yanjie Zhao, Xinyi Hou, Shenao Wang, and Haoyu Wang. 2024. Llm app store analysis: A vision and roadmap. *ACM Transactions on Software Engineering and Methodology* (2024).

[51] Chijin Zhou, Mingzhe Wang, Jie Liang, Zhe Liu, Chengnian Sun, and Yu Jiang. 2019. Visfuzz: Understanding and intervening fuzzing with interactive visualization. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1078–1081.

[52] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* (2024).